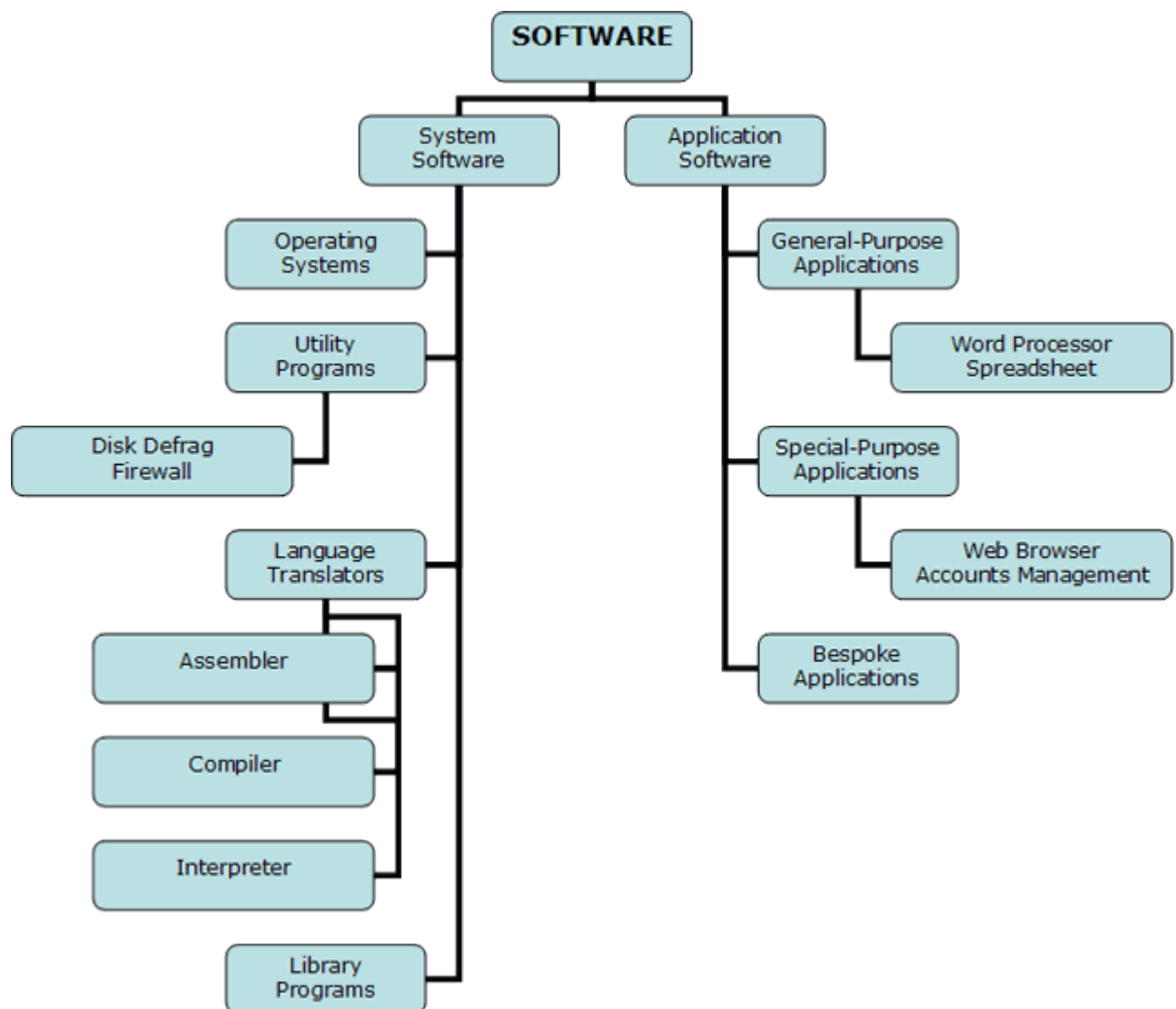


Computer Science

Classification Of Software

The Big Picture

The following diagram shows the way we classify software.



System Software

We use the term **System Software** for software which is primarily used to operate the hardware.

Operating Systems

The operating system is the software that allows you to operate the hardware. The programs that we want to execute, the applications that we want to use all require a platform on which to execute. That platform is provided by the operating system.

One role of the operating system is to **provide a virtual machine**. This refers to the way that, by clicking on icons and menus, or by typing in commands at a prompt, we get to interact with the computer hardware without having to understand its complexity. By hiding the true complexity of the system from the user, the operating system makes it easier for ordinary people to make computers perform useful tasks.

Utility Programs

Some utility programs are bundled in with operating system software these days. Others you buy or source in some other way.

Utility programs tend to perform specific tasks related to the management of hardware. Examples of utility programs include compression programs, formatters, defragmenters and other disk management tools.

Library Programs

Library programs are compiled libraries of commonly-used routines. On a Windows system they usually carry the file extension **dll** and are often referred to as **run-time libraries**. The libraries are run-time because they are called upon by running programs when they are needed. When you program using a run-time library, you typically add a reference to it either in your code or through the IDE in which you are programming.

Some library programs are provided within operating systems like Windows or along with development tools like Visual Studio. For example, it is possible to download and use a library of routines that can be used with Windows Media Player. This includes things like making playlists, functions and procedures for accessing and manipulating the music library (which is a binary file) and playback routines.

Using library programs saves time when programming. It also allows the programmer to interact with proprietary software without having access to its source code.

Language Translators

Whatever language or type of language we use to write our programs, they need to be in machine code in order to be executed by the computer. There are 3 main categories of translator used,

Assembler

An assembler is a program that translates the mnemonic codes used in assembly language into the bit patterns that represent machine operations. Assembly language has a one-to-one equivalence with machine code, each assembly statement can be converted into a single machine operation.

Compiler

A compiler turns the source code that you write in a high-level language into object code (machine code) that can be executed by the computer.

The compiler is a more complex beast than the assembler. It may require several machine operations to represent a single high-level language statement. As a result, compiling may well be a lengthy process with very large programs.

Interpreter

Interpreters translate the source code at run-time. The interpreter translates statements one-at-a-time as the program is executed.

Interpreters are often used to execute high-level language programs whilst they are being developed since this can be quicker than compiling the entire program. The program would be compiled when it is complete and ready to be released.

Interpreters are also used with high-level scripting languages like PHP, Javascript and many more. These instructions are not compiled and have to be interpreted either by the browser (in the case of Javascript) or by interpreters on the server (in the case of PHP).

Some programming languages make use of both compilers and interpreters. If you were to write a Java program in a text editor, when you came to compile it with the Java compiler, you would actually be creating something called **bytecode**. Bytecode can be thought of as an intermediate stage between source code and object code. When a computer executes a Java program, library programs on that machine interpret the bytecode. This allows Java to be platform-independent - a user needs the correct run-time libraries for Java on their machine in order to execute the programs.

Application Software

Application software tends to be used for the tasks that have some relationship to the world outside of the computer. For example, you might use a word processor to write a letter or an essay. Although you use the computer to perform the task, the task itself might reasonably be considered to be a non-computer task.

General-Purpose Software

Software is general-purpose if it can be used for lots of different tasks. You can use a word processor to write letters, memos, essays, instructions, notes, faxes, invoices and lots more.

These days we tend to use integrated suites of office software where a range of general-purpose software is provided, usually with the facility to combine elements from each application in a single file.

Special-Purpose Software

This software performs a single specific task. This task might be complex like payroll calculation, stock control etc. but will be based on a single task.

As with many abstract concepts, you can stretch these definitions until they blur a little. These days, web browsers can contain a lot of features. They are still primarily focused on a single task, rendering web pages and so the web browser is special-purpose. Being able to access an application using a browser does not change the main purpose of the browser software itself.

Bespoke Software

Bespoke software is written for a single client. Large organisations have a need for well-developed applications suited to their specific needs. Such software is often expensive to develop since the development costs are not shared among a large number of people purchasing the software.

Generations Of Programming Language

First Generation Languages

Here we are talking about machine code. This is the only form of code that can be executed by the computer directly.

Second Generation Languages

Assembly language was developed to make it easier for programmers to write instructions than it would be using machine code. Mnemonics are used instead of bit patterns (which are harder to remember).

First and second generation languages are **low level** and **machine-oriented**. This refers to the way that they are based on the machine operations that are available for a given processor.

Third Generation Languages

Third generation languages are **high level**, **platform-independent** and **problem oriented**. When source code is compiled, there is a one-to-many equivalence of high level language statements to machine code statements. Third generation programs can be run on any platform for which an appropriate compiler or interpreter exists.

High level languages are developed to help solve particular types of problem. The FORTRAN language was designed with Mathematics, Science and Engineering in mind, it contains lots of scientific functions that the average programmer may not need. The COBOL language was developed with business logic in mind, PHP was developed for server-side scripting and so on.

All of the languages in the first 3 generations are called **imperative** languages because the program's statements are executed in the order specified by the programmer.

Fourth Generation Languages

Fourth generation languages are **declarative**. This means that the programmer will write facts or rules rather than statements. The interpreter for the language produces the result using whichever standard algorithms it has been given for doing so.

SQL and Prolog are both examples of declarative languages. Both are described in the programming section of this site and are relatively easy to try out. A quick half-hour blast at each would give you a feel for how they work and help you to understand how they differ from the other types of language.

Pages designed and coded by MHA since 2003 | Valid [HTML 4.01](#)(Strict) | [CSS](#)